Table des matières

Cours 12 — Les codes 1

Cours 12 — Les codes

Cours 12/12 · La Brûlerie — Formation Data Engineering

Sources	**Ingestion**	**RAW**
d'où vient la donnée	aller la chercher, et la charger	la déposer, la ranger, la traiter
✓ ici · De bout en bout, les deux chemins, montés en entier	✓	✓

À ouvrir après avoir essayé. Le code garde les commentaires numérotés du cours, pour comparer ligne par ligne. `pipeline.py` est montré en entier, dans son nouvel état ; le `README.md` est un exemple, que vous écrivez dans vos mots.

01_de/reconstruire.cmd

```
@echo off
rem 01_de/reconstruire.cmd : rebatir les deux chemins de zero, en une commande, depuis Projet_Brulerie
rem 1. les dossiers du bronze et de l'entrepot : ni COPY ni DuckDB ne creent ceux du dessus
if not exist plateforme\lac\bronze mkdir plateforme\lac\bronze
if not exist plateforme\entrepot mkdir plateforme\entrepot
rem 2. le chemin froid : le bronze, la RAW et l'argent jusqu'au 30 juin, juges par leurs regles
python 01_de\pipeline.py 2025-06-30 || exit /b 1
rem 3. le chemin chaud : le journal du direct, relu depuis son decalage, l'alerte a 218 degres
python 01_de\consommer.py 218 || exit /b 1
echo Reconstruction : au vert.
```

01_de/pipeline.py

```
# 01_de/pipeline.py : la porte d'entrée du pipeline, qui enchaîne les étapes et rend un code de sortie
import sys
import duckdb
from charger import charger
from controler import controler

def bronze(jusqua):
    # 1. le bronze, refait en entier ; son contrôle se juge désormais avec les règles de la qualité
```


© 2026 iAlexMG. Tous droits réservés.

```
    source, tickets = duckdb.sql(open("01_de/sql/bronze.sql", encoding="utf-8").read()).fetchone()
    return f"{tickets} tickets rangés, {source} à la source"

def raw(jusqua):
    # 2. la RAW, chargée jusqu'au jour donné ; puis l'argent, refait en entier à partir d'elle
    with duckdb.connect("plateforme/entrepot/brulerie.duckdb") as con:
        charger(con, jusqua)
        con.execute(open("01_de/sql/argent_torrefaction.sql", encoding="utf-8").read())
        return f"{con.execute('SELECT COUNT(*) FROM journal').fetchone()[0]} fichiers au journal"

def lancer(jusqua):
    # 3. chaque étape tracée ; la première qui échoue arrête tout, et le code de sortie vaut 1
    for nom, etape in (("bronze", bronze), ("raw", raw), ("controles", controler)):
        try:
            print(f"{nom} : {etape(jusqua)}")
        except Exception as erreur:
            print(f"{nom} : échec, {erreur}")
            return 1
    return 0

if __name__ == "__main__":
    sys.exit(lancer(sys.argv[1]))
```

01_de/README.md

```
# 01_de — les données de La Brûlerie du Passage

<!-- 1. ce qu'est le dépôt, et ce qu'il livre -->
Le pipeline de données d'un café de quartier de Montréal, qui torréfie son café sur place. Il range
les tickets de la caisse en bronze, dans un lac de fichiers Parquet ; il charge la caisse et le
torréfacteur dans la `RAW` d'un entrepôt DuckDB, tout en texte ; il tire l'argent des lots de leurs
courbes ; il lit le direct du torréfacteur et lève une alerte de température. Quatre règles de la
qualité jugent chaque passage. Juin 2025 retombe sur la caisse, au cent : 1 511 ventes payées,
16 718,00 $ avant taxes.

## Ce qu'il faut avoir

<!-- 2. ce qui n'est pas dans le dépôt : les sources, le journal du direct, les outils -->
- `Projet_Brulerie/sources/`, les fichiers du café, intacts ;
- `Projet_Brulerie/plateforme/direct/journal.jsonl`, le journal du direct, qu'écrit le rejoueur
  fourni ;
- l'environnement conda `formations` : Python 3.11, DuckDB 1.5 et pytest.
```


© 2026 iAlexMG. Tous droits réservés.

Rebâtir

<!-- 3. la commande, d'où la lancer, et ce qui doit sortir -->
Depuis `Projet_Brulerie`, sur un projet qui n'a que ses sources et le journal du direct, ou dont on a effacé le lac, `plateforme\direct\decalage.txt` et les tables de Data Engineering de l'entrepôt partagé : `01_de\reconstruire.cmd`. Elle monte le chemin froid, puis le chemin chaud, et finit par `Reconstruction : au vert.`, code 0. Relancée, elle ne double rien.

Faire tourner et vérifier

<!-- 4. une nuit, un rattrapage, les règles, le test -->
- une nuit : `python 01_de\pipeline.py <jour>`, code 0, ou 1 dès qu'une étape échoue ou qu'une règle à bloquer est rompue ;
- un rattrapage : `python 01_de\rattraper.py <premier jour> <dernier jour>`, arrêté au premier jour en échec ;
- les règles : `sql/controles.sql`, jugées par `controler.py` à chaque passage ;
- le test : `pytest 01_de`, qui prouve que relancer le chargement ne double rien.

© 2026 iAlexMG. Tous droits réservés.